



ISEL
INSTITUTO SUPERIOR DE
ENGENHARIA DE LISBOA

SVLC: Service virtualization based on Linux Containers

Daniel Carvalho

Supervisors: Luis Osório
Mário Pinheiro

Project report carried out within the scope of Project and Seminar.
Degree in IT and Computer Engineering.

June, 2024

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

SVLC: Service virtualization based on Linux Containers

Student: 49419 - Daniel Martinho de Carvalho

Supervisores: António Luís Freixo Guedes Osório

Mario Justiniano Morais Pinheiro

Project report carried out within the scope of Project and Seminar.
Degree in IT and Computer Engineering.

Resume

The Linux container concept has experienced a notable surge in adoption over the past decade, largely propelled by the widespread adoption of Docker. As organizations increasingly seek efficient and scalable solutions for application deployment and management, Linux containers have emerged as a pivotal technology, offering lightweight, portable, and isolated environments for running applications.

In this project, our goal is to develop a comprehensive set of functionalities to facilitate the instantiation, management, and deployment of applications within Linux containers. Furthermore, we aim to extend these capabilities to the Kubernetes cluster concept—a leading container orchestration platform—enabling seamless integration with cloud-native architectures.

Aligned with the principles of open collaboration and standardization, our project embraces open-source initiatives, particularly those endorsed by the Linux Foundation’s Open Container Initiative (OCI). By leveraging open standards and community-driven development, we seek to foster interoperability, transparency, and innovation within the container ecosystem.

Acknowledgements

I would like to express my sincere gratitude to Engineers Luis Osório and Mário Pinheiro for their continuous support and valuable guidance throughout this project, and to Engineer Carlos Gonçalves for his assistance in understanding the ISOS framework. Special thanks to Engineer Rui Bernardo from Claranet for accepting the challenge of being this project's jury. I am also grateful to my colleagues and friends for their encouragement and insightful discussions throughout the project. Finally, I express my heartfelt thanks to my family for their unconditional love and unwavering support throughout this journey.

Table of Contents

1	Introduction	9
2	Background Knowledge	12
2.1	Informatics System of Systems (ISoS)	13
2.2	Linux Containers and Podman	14
2.3	Kubernetes Technology Framework	15
2.4	Continuous Integration and Continuous Deployment (CI/CD)	16
2.4.1	Continuous Integration	17
2.4.2	Continuous Deployment	17
2.5	CI/CD and ISoS Integration	17
2.6	Application Packaging (ISystem) in Kubernetes	17
2.6.1	Helm Package Manager	17
3	SVLC Architecture and Implementation	18
3.1	Problem Explanation:	18
3.2	Strategy to Solve the Problem:	18
3.3	Implementation in a Pod	20
3.3.1	PodmanCmd Class	20
3.3.2	PodmanHttp Class	21
3.3.3	Considerations for Simple Service Elements	21
3.3.4	Conclusion	21
3.4	Implementation in a Kubernetes Cluster	22
3.4.1	Kubernetes Class	22
3.4.2	Considerations for Service Elements	23
3.4.3	Conclusion	23
3.5	Result Handling	23
3.6	Respecting CI/CD Concepts	24
3.7	Challenges and Solutions	25
3.8	Summary	25

4	Conclusion and Future Directions	26
4.1	Summary of Findings	26
4.2	Future Work	27
4.3	Conclusion	27
	References	29

List of Figures

1.1	Project idea.	11
2.1	Example of an ISystem.	14
2.2	Example of a Pod.	15
2.3	Example of a single node.	16
2.4	Example of a Kubernetes cluster.	16
3.1	Implementation overview	20

Chapter 1

Introduction

The rapid evolution of technology demands efficient and scalable solutions for application deployment and management. An application refers to the technological artifacts, code, dependencies, and configurations needed to make it work. Traditional computer machine virtualization techniques often incur substantial resource and operational overhead, leading to the search for more efficient alternatives. This has contributed to the rise of containerization technologies, with Linux containers becoming mainstream largely due to the success of Docker [1].

Docker, introduced in 2013, revolutionized application development by packaging applications and their dependencies into immutable images. These images, when executed, create portable and consistent containers that run uniformly across various environments [2]. The concept of containerization can be likened to creating a "bunker" for applications. This bunker encapsulates the application and all its dependencies, providing an isolated and secure environment that is impervious to changes and inconsistencies in the underlying infrastructure.

The portability of containers is achieved through the immutability of the images, ensuring that the same environment is reproduced wherever the container is deployed. This innovation has driven the widespread adoption of containers, offering lightweight, isolated, and repeatable application environments that mitigate the issues of resource and operational overhead associated with traditional virtualization.

While initially focused on Linux environments, container technology has also been extended to Windows OS. Windows containers provide similar benefits, allowing applications to be packaged with their dependencies and run consistently across different Windows environments [3]. This expansion to Windows has broadened the applicability of containerization, enabling developers to use containers for a wide range of applications across diverse operating systems, further driving the adoption and utility of container technology [4].

With the widespread adoption of containers, managing distributed services has become increasingly complex [5]. Container orchestration platforms like Kubernetes simplify this by facilitating the automation of deployment, scaling, and operation of containerized applica-

tions, making them vital for modern cloud-native architectures [5]. These platforms play a crucial role in Service Virtualization based on Linux Containers (SVLC), enabling efficient management and control of containerized services to ensure smooth operation within the broader system.

Kubernetes was developed to meet the need for efficient distributed services distribution. It addresses the complexities of deploying, managing, and scaling services across a distributed infrastructure, ensuring that various components of an informatics system work together seamlessly and reliably. By providing a unified platform for managing containers across multiple nodes in a cluster, Kubernetes ensures that containerized services can communicate and cooperate effectively, regardless of their deployment location. This orchestration automates many manual tasks associated with managing containerized applications, thereby reducing operational complexity and enhancing the scalability and reliability of the entire system.

This project addressed the following key challenges:

- Simplifying the creation, management, and deployment of Informatics Systems (ISystems). An ISystem can be compared to a file system where the root is the ISystem, the Cooperation Enabled Services (CES) are directories, and the Services are files, containing the data.
- Enhancing scalability and reliability through seamless integration with Kubernetes. This addresses the need to distribute Service elements or even parts of a Service element across different computers efficiently. This capability is essential for managing, deploying, and scaling containerized applications.
- Ensuring compatibility and interoperability among different container tools and platforms by adhering to Open Container Initiative (OCI) standards [6]. This adherence promotes transparency and reduces vendor lock-in through standardized formats and runtimes.

By tackling these challenges, the aim is to advance containerization technology and provide practical solutions that benefit the broader community.

The following diagram (Figure 1.1) illustrates the role of SVLC (Service Virtualization based on Linux Containers) as middle service. SVLC manages the provisioning and deployment of services to Podman pods or Kubernetes clusters, ensuring all dependencies are configured, organized, and prepared for seamless deployment.

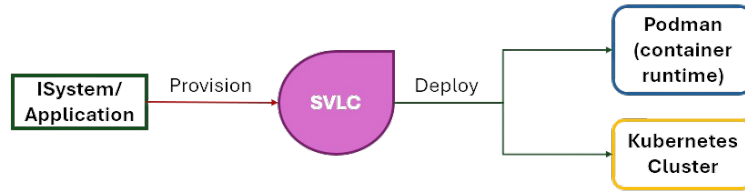


Figure 1.1: Project idea.

The remainder of this report is structured as follows:

- **Chapter 2** - Review of existing container concepts, orchestration platforms, and theoretical concepts essential for understanding the project.
- **Chapter 3** - Design and implementation of the container management and deployment functionalities, along with Kubernetes integration.
- **Chapter 4** - Performance and scalability evaluation through tests and benchmarks.
- **Chapter 5** - Summary of findings, contributions, and suggestions for future work.

Chapter 2

Background Knowledge

The instantiation of multiple applications on the same computer or operating system presents critical challenges, particularly in terms of isolation and resource management. This issue is exacerbated when applications are not designed to share resources efficiently or securely. The advent of **Linux containers** and **container clusters** has become a pivotal solution in addressing these challenges within the realm of cloud computing.

Linux containers are lightweight, isolated environments that package applications and their dependencies, ensuring consistent deployment across various computing environments and enhancing deployment efficiency.

Container clusters, managed by orchestration platforms like Kubernetes, are interconnected groups of containers that automate deployment, scaling, and management tasks, enabling efficient orchestration of large-scale applications.

Cloud computing offers computing services—servers, storage, databases, networking, software, analytics, and intelligence—over the Internet, fostering rapid innovation, flexible resources, and economies of scale[7].

The concept of an **Informatics System of Systems (ISoS)** emerges from the need to abstract computational artifacts and their aspects, focusing on managing and understanding interconnected systems of computing technology artifacts.

In the era of cloud computing, the scale of users and data necessitates a lighter infrastructure capable of coordinating distributed application parts across a network of operating systems. A *lightweight operating system*, or a **container**, contains only the essential resources needed for executing each service component of an application [1]. Although a container can encapsulate multiple service elements of an ISystem, it is advisable to ensure that each component (service element) within the container receives only the necessary resources for its operation [2]. The network of containers, based on a Kubernetes cluster that resembles distributed operating system abstraction concepts [8] for managing these containers, ensures reliable application performance in diverse environments.

The remainder of the chapter is organized into the following sections:

1. Informatics System of Systems (ISoS);
2. Linux Containers and Podman;
3. Kubernetes Technology Framework;
 - Node Structure;
 - Kubernetes Cluster.
4. Continuous Integration and Continuous Deployment (CI/CD);
 - Continuous Integration;
 - Continuous Deployment.
5. Application Packaging (ISystem) in Kubernetes.
 - Helm Package Manager.

2.1 Informatics System of Systems (ISoS)

The ISoS framework addresses the need for a unified approach to integrate diverse technological components within one or more applications. In this context, when we refer to "application," we are specifically talking about the technological artifacts and dependencies necessary for the execution of a particular component or system that the ISoS model encapsulates into Service abstraction. These artifacts and dependencies include everything needed to run and support that Service element, such as libraries, frameworks, configuration files, and any other software or resources required. This definition of "application" focuses on the entirety of technological elements that enable the functionality and operation of the component being discussed. The framework provides an integration model that supports multiple suppliers or vendors, ensuring compatibility and interoperability across different technological landscapes [9].

At its core, an ISoS comprises a hierarchical structure of interconnected components: ISystems, Cooperative Enabled Services (CES), and individual Services (see Figure 2.1). In this hierarchy, the computational logic and necessary resources for execution are contained within Service elements. CES elements logically group of one or more Services, and an information system (ISystem) is composed of one or more CES elements. Together, they encapsulate the computational logic and essential resources required for application execution. By leveraging the container model and encapsulating technological artifacts within service elements, ISoS enables the deployment of isolated and independent applications [10]. This approach promotes modularity, scalability, and flexibility, essential qualities in modern software development environments.

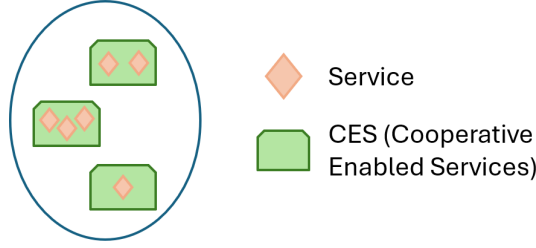


Figure 2.1: Example of an ISystem.

2.2 Linux Containers and Podman

Linux containers encapsulate a set of one or more processes and their associated resources in isolation. These containers obtain all required files from a specific image, ensuring portability and consistency throughout different stages of development and production. This characteristic enables faster deployment compared to traditional "development pipelines that rely on replicating testing environments" [11].

Podman, short for Pod Manager, is a lightweight container management tool that has gained prominence as an alternative to Docker. Developed from Docker's open-source code, Podman initially retained compatibility with Docker, adhering to an open-source software (OSS) model, particularly in its foundational functionalities. However, over time, Docker has introduced new features that differentiate it from Podman. The Podman-desktop version offers a user-friendly interface for managing containers and pods, providing functionalities for creating, running, and managing containerized applications. Podman embraces the principles of containerization, enabling users to deploy and manage containers efficiently across diverse computing environments [12] that supports a compatible container runtime.

The concept of a Pod, within the Kubernetes infrastructure, refers to an isolated group that includes one or more containers (see Figure 2.2). Containers within a Pod share the same namespaces and cgroups (resource constraints), ensuring isolation for system resources such as process IDs, network access, and file systems. This setup prevents interference between containers within the same Pod or with other Pods. Notably, Pods also share the same localhost, and an IP address is assigned to the Pod as a whole, rather than to individual containers within it. When a container is added to a Pod, Podman integrates its processes into the shared cgroups and namespaces. Moreover, each container within a Pod features a container monitor process known as *conmon* [13]. The *conmon* process is responsible for managing the container's lifecycle and capturing its standard input, output, and error streams. It communicates with Podman or another container runtime to relay information about the container's status, resource usage, and events. This monitoring ensures that containers within the Pod operate correctly and are properly managed throughout their lifecycle.

To illustrate, consider previous research work on operating system isolation spaces around the beginning of the eighties (1979/1982), such as the chroot system call [14]. *Chroot* changes

the root directory of a process and its children to a new location in the file system, creating a kind of "bunker", a new root directory for initiating a new instance of a lightweight operating system, a contained environment. This restricts the process to interact only with files within the new root directory, similar to how containers isolate their file systems based on namespace and cgroup Linux kernel mechanisms. For example, by setting the *chroot* to `/var/mycontainer`, access is confined to `/`, thereby ensuring that applications operate within a controlled environment similar to a new instance of an operating system, complete with a new init process (the Linux operating system's init process).

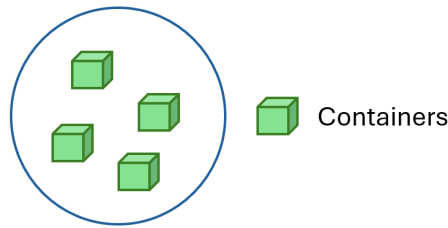


Figure 2.2: Example of a Pod.

2.3 Kubernetes Technology Framework

Kubernetes, a container orchestration platform, has become pivotal in modern software development. It automates application deployment, scaling, and management across diverse environments, enhancing agility and efficiency. By streamlining operations, Kubernetes enables faster, more reliable, and scalable application delivery, marking a significant advancement in software deployment practices [15]. As containerized applications grow in use, the Kubernetes framework can influence system development significantly by shaping how elements of informatics systems are instantiated, thanks to its capabilities for automated container deployment, scaling, and management [16]. Its significance lies in adopting a declarative approach, facilitated through manifest files (YAML). Unlike imperative methods that specify how to achieve a desired state, a declarative approach defines the desired state of the application and infrastructure. Kubernetes interprets these manifest files to automatically provision and manage resources according to the specified configurations. This approach simplifies operations, improves consistency, and reduces errors in managing complex containerized environments.

The Kubernetes orchestrator boasts several essential runtime features that contribute to its effectiveness. These include service discovery, which provides stable DNS names and IP addresses to deployed services, facilitating seamless communication between components. Additionally, Kubernetes supports horizontal scaling, enabling applications to dynamically adjust resources based on workload demands. Load balancing functionality optimizes resource allocation and traffic distribution, enhancing performance and reliability.

Moreover, Kubernetes incorporates self-healing mechanisms to ensure high availability and fault tolerance; for instance, if a pod fails, Kubernetes can automatically restart it or replace it with a new instance to maintain the desired state of the service element. In the event of container failures, Kubernetes automatically restarts failed containers and redirects traffic away from temporarily unavailable instances. Configuration management features further streamline deployment processes, allowing for centralized configuration and efficient resource utilization [16].

Node Structure

A Node represents a single machine that can host one or more pods. These nodes can be part of a Kubernetes cluster if multiple pods are present (see Figures 2.3 and 2.4). In a production environment, a node is typically either a physical machine or a virtual machine hosted in the cloud [17].

Kubernetes Cluster

A Kubernetes cluster is a network of computer nodes (physical or virtual) that communicate and share resources, much like a honeycomb cluster (see Figure 2.4). The cluster and Kubernetes handle distributing work to those nodes. They also ensure that if a node is added or removed from the cluster, work will be shifted around as necessary [17].

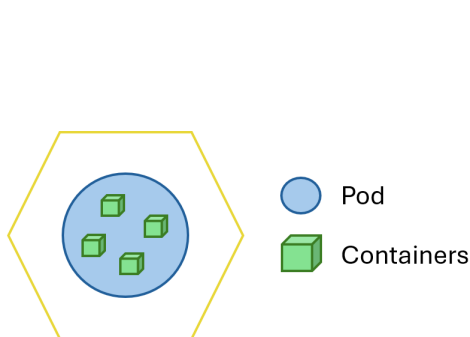


Figure 2.3: Example of a single node.

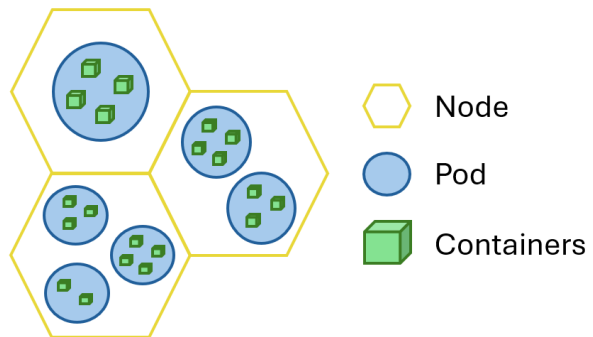


Figure 2.4: Example of a Kubernetes cluster.

2.4 Continuous Integration and Continuous Deployment (CI/CD)

CI/CD automates code integration and deployment for rapid, reliable software releases [18]. Embracing CI/CD enhances efficiency, reduces errors, accelerates time-to-market, and ensures deployment consistency across teams.

Transitioning from traditional infrastructure to Kubernetes is contributing to software development. Kubernetes' containerization streamlines deployment, improves resource utilization, and integrates seamlessly with CI/CD practices. This transformation boosts agility, scalability, and reliability while simplifying operations and accelerating time-to-market.

2.4.1 Continuous Integration

Continuous Integration involves the automated merging of code changes from multiple contributors into a shared repository. Tools such as Jenkins, Travis CI, and CircleCI facilitate this process by running automated tests and validating code changes to ensure they do not break the existing codebase [19].

2.4.2 Continuous Deployment

Continuous Deployment extends CI by automatically deploying code changes to production after they pass the automated tests. This practice minimizes manual intervention and reduces the time between development and release, leading to more efficient and frequent updates [20].

2.5 CI/CD and ISoS Integration

Integrating CI/CD practices with the ISoS framework transforms software development by streamlining the integration, testing, and deployment of diverse technological components within applications. ISoS promotes modularity, scalability, and flexibility by encapsulating artifacts within service elements and leveraging containerization for isolated application deployments.

Together, CI/CD and ISoS foster agility, scalability, and reliability in software development, aligning with modern standards to meet the dynamic demands of technology landscapes.

2.6 Application Packaging (ISystem) in Kubernetes

Application packaging in Kubernetes revolves around bundling applications and their dependencies into a deployable unit. This process simplifies deployment and management, enhancing scalability and consistency within Kubernetes environments. A key tool for managing Kubernetes applications is Helm.

2.6.1 Helm Package Manager

Helm is a widely adopted package manager for Kubernetes, streamlining the deployment and management of applications. It utilizes a packaging format known as charts, which consist of files defining a cohesive set of Kubernetes resources [21]. These charts encapsulate everything needed to deploy an application, including Kubernetes manifests, configurations, and dependencies.

Chapter 3

SVLC Architecture and Implementation

The Service Virtualization based on Linux Containers (SVLC) suite builds on the ISoS framework to streamline the development and deployment of ISystems images. By implementing CI/CD (continuous integration and continuous delivery) practices, we automate the manual tasks traditionally required to move new code from a commit into production. This includes the build, test (integration tests, unit tests, and regression tests), and deployment phases, as well as infrastructure provisioning [22].

3.1 Problem Explanation:

In an Informatics System, multiple Service Elements encapsulate critical business logic and resources. To facilitate efficient hosting and deployment, these elements must be encapsulated within container images. The challenge also involves provisioning and configuring the deployment environment to ensure optimal deployment and execution.

3.2 Strategy to Solve the Problem:

The Service Virtualization based on Linux Containers (SVLC) tools suite, built on the ISoS framework, acts as middleware between ISystem_0 and the deployment of ISystems or Services. SVLC manages the provisioning of technological artifacts, ensuring all dependencies are configured, organized, and prepared for seamless deployment. This approach includes crucial infrastructure preparation, such as setting up a Kubernetes network to ensure it is ready to receive deployment payloads. By doing so, SVLC ensures that the underlying infrastructure is properly configured to support the deployment and operation of containerized systems.

- **Simplify Container Management:**

- *Objective:* Reduce operational complexity.
- *Strategy:* Automate the build, testing (unit, integration, and regression tests), and deployment phases using CI/CD practices, ensuring infrastructure readiness, including preparing a Kubernetes network to receive deployment payloads.

- **Enhance Scalability and Reliability:**

- *Objective:* Improve container orchestration by automating deployment, management, and scaling for efficient, seamless, and resilient application operation.
- *Strategy:* Integrate with Kubernetes to efficiently deploy, manage, and scale containerized applications in a prepared network environment.

- **Ensure Compatibility and Interoperability:**

- *Objective:* Promote transparency and reduce vendor lock-in in relation to platform and container technology choices.
- *Strategy:* Adhere to Open Container Initiative (OCI) standards to ensure compatibility across different platforms, aligning with the prepared Kubernetes network infrastructure.

In an Informatics System, modeled as a tree with **ISystem** as the root, **CES** as intermediate nodes, and **Service Elements** as the leaves, implementing these strategies simplifies the creation, management, and deployment of containerized **Service Elements**. This enhances the scalability, reliability, and interoperability of the system. For simplicity, consider an **ISystem** as a composition of **Service Elements**.

Deployment mechanisms were considered for instantiating **ISystems** while managing dependencies and configurations see Figure 3.1, ensuring a seamless deployment process. This guarantees that once deployed, the services require no additional preparations to function correctly.

The design and implementation are illustrated using two simple Spring servers as examples: a **hello-world** server and a **goodbye-world** server. Initially, these **Service Elements**, are deployed as monolithic applications on a single pod each using Podman. As the project transitions to a distributed architecture, this two **Service Elements** are deployed separately within a Kubernetes cluster. Each service is encapsulated in a pod, including the container and necessary configurations. Kubernetes efficiently manages, scales, and orchestrates these services, ensuring independent communication, scaling, and updates. This demonstrates the scalability and flexibility of containerization and orchestration in a distributed system.

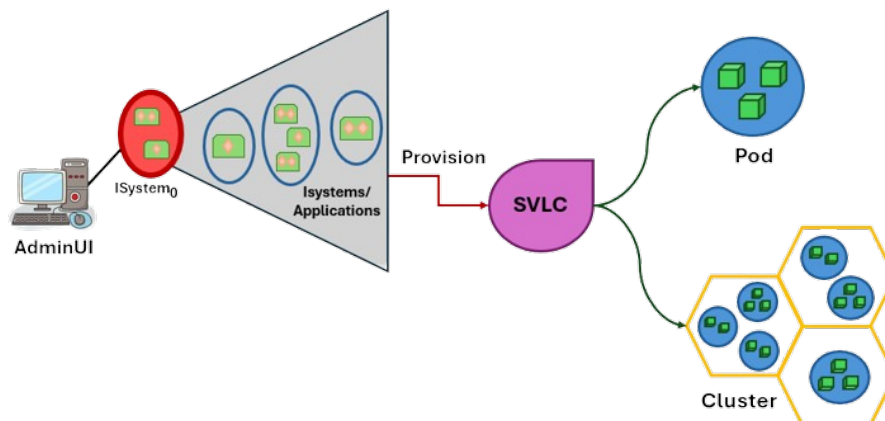


Figure 3.1: Implementation overview

The remainder of the chapter is organized into the following sections:

1. Implementation in a Pod;
2. Implementation in a Kubernetes Cluster;
3. Challenges and Solutions;
4. Summary.

3.3 Implementation in a Pod

SVLC leverages Podman for container management through both the command-line interface (CLI) and HTTP requests. This integration is encapsulated in two main classes: `PodmanCmd` and `PodmanHttp`, which implement the `Podman` interface. This design allows for flexible interaction with Podman, enabling operations such as creating, starting, stopping, and managing pods and containers, as well as pulling and deleting images.

3.3.1 PodmanCmd Class

The `PodmanCmd` class is responsible for executing Podman commands via the CLI. It utilizes the `CmdExec` utility class to run these commands in a process, capturing the output or errors. This class supports a variety of operations:

- **Creating Pods and Containers:** Commands like `podman pod create` and `podman create` are constructed and executed.
- **Managing Pods:** Starting, stopping, and pruning pods are handled through respective Podman commands.
- **Image Management:** Pulling new images or deleting existing ones is performed via commands like `podman pull` and `podman image rm`.

This class works in any environment, Linux or Windows, as long as Podman is installed on the machine.

The `CmdExec` utility class, used by `PodmanCmd`, supports both synchronous and asynchronous command executions. Synchronous executions return the output directly, while asynchronous executions return the thread of the concurrent process, which can be utilized to terminate the command.

3.3.2 PodmanHttp Class

The `PodmanHttp` class interacts with Podman's REST API to manage containers and pods. It uses the `HttpReq` utility class to send HTTP requests to the Podman server. This class supports similar operations as `PodmanCmd` but through HTTP requests:

- **Creating and Managing Pods:** HTTP POST requests are sent to endpoints like `/pods/create` and `/pods/{podName}/start`.
- **Image Management:** Pulling images is done by sending a POST request to `/images/pull`, with authentication handled via headers if necessary.

Note that the `PodmanHttp` class only works in a Linux environment. Additionally, it requires Podman to be running as a service, which can be enabled using the command `podman system service tcp:{ip}:{port}`. By default, this service will run for 5 seconds if the `--time` flag is not specified, and for an endless duration if set to 0. The class constructs JSON payloads for requests that require them and handles responses.

The `PodmanHttp` class uses the `CmdExec` utility class to execute the `podman system service` command asynchronously, allowing for concurrent processing.

3.3.3 Considerations for Simple Service Elements

This implementation is designed for simple service elements that only need to export a port in order to work. It is required that these service elements have an image available in any registry.

3.3.4 Conclusion

Integrating Podman into our Java application through both CLI and HTTP interfaces provides a flexible and powerful way to manage containers and pods. This design allows our application to interact with Podman in a manner that suits various deployment environments, from local development to cloud-based services, ensuring our application's components are managed efficiently and reliably.

3.4 Implementation in a Kubernetes Cluster

SVLC leverages Kubernetes for managing and orchestrating containerized services across a cluster of nodes. This integration is encapsulated in the `Kubernetes` class, which handles various aspects of Kubernetes resource management through a comprehensive set of methods. This design allows for flexible and scalable interaction with Kubernetes, enabling operations such as listing resources, scaling deployments, retrieving logs, enabling autoscaling, and deploying resources from YAML definitions.

3.4.1 Kubernetes Class

The `Kubernetes` class provides a unified interface for interacting with Kubernetes. It utilizes the Kubernetes API, facilitated by the `kubernetes-client` library [23], to perform the following operations:

- **Listing Pods:** The `listPods()` method retrieves and returns a list of all pods in the specified namespace.
- **Listing Deployments:** The `listDeployments()` method returns a list of all deployments in the specified namespace.
- **Listing Services:** The `listServices()` method returns a list of all services in the specified namespace.
- **Scaling Deployments:** The `scaleDeployment(String deploymentName, int replicas)` method scales the specified deployment to the desired number of replicas.
- **Retrieving Pod Logs:** The `getPodLogs(String podName)` method fetches and returns the logs of the specified pod.
- **Enabling Autoscaling:** The `enableAutoscaling(String deploymentName, int minPods, int maxPods, int targetCPUUtilizationPercentage)` method sets up autoscaling for the specified deployment with the given parameters.
- **Getting Node States:** The `getNodeStates()` method retrieves the status of all nodes in the Kubernetes cluster and returns it as a map.
- **Getting a Node State:** The `getNodeState(String nodeName)` method retrieves the status of a specific node in the Kubernetes cluster.
- **Deploying from YAML:** The `deployFromYaml(String yaml)` method deploys a Kubernetes resource defined in a YAML string and returns the deployed resource.

3.4.2 Considerations for Service Elements

This implementation is designed for more complex service elements that require management and orchestration within a distributed environment. It is essential that these service elements have container images available in a registry accessible by the Kubernetes cluster.

3.4.3 Conclusion

Integrating Kubernetes into SVLC through the `Kubernetes` class provides a scalable and powerful way to manage containers and services across a distributed infrastructure. This design allows our application to interact with Kubernetes efficiently, ensuring that the components are managed and orchestrated to meet the demands of modern cloud-native environments.

3.5 Result Handling

The `Kubernetes` class, along with `PodmanCmd` and `PodmanHttp`, utilize the `Result` class indirectly for operation outcomes. All methods in these classes are designed to conform to the `DefaultProc` functional interface, ensuring a more generic and flexible approach. This setup allows any command to be executed through the `CommonExec` class, which standardizes the handling of success and error results.

The `DefaultProc` interface is a functional interface that represents a function receiving no arguments and returning a value. By encapsulating methods in the `PodmanCmd`, `PodmanHttp`, and `Kubernetes` classes within this interface, we achieve a high level of abstraction and reusability.

The `CommonExec` class provides a static method `exec` that takes a `DefaultProc` as an argument. This method executes the given command, returning a `Result` object. Even though this class is generic, it specifically represents either a `Failure` class with an error message or a `Success<T>` class with any value of type `T`. In case of an error, a `Failure` object is returned, encapsulating the error message. In case of successful execution, a `Success<T>` object is returned, containing the resultant value.

This design pattern is crucial for chaining operations and handling errors gracefully within the application. It ensures robust and flexible management of operation outcomes, allowing for streamlined and error-resistant execution flows.

In summary, the `Result` handling mechanism used by the `PodmanCmd`, `PodmanHttp`, and `Kubernetes` classes ensures robust and flexible management of operation outcomes. By leveraging the `DefaultProc` interface and the `CommonExec` class, the application achieves a streamlined and error-resistant execution flow.

3.6 Respecting CI/CD Concepts

This design aligns with CI/CD (Continuous Integration/Continuous Deployment) principles in the following ways:

- **Automation of Builds and Deployments:** The integration of Podman allows for the automated creation, management, and deployment of containers, which is a core aspect of CI/CD pipelines.
- **Environment Consistency:** By using containerization, our application ensures consistent environments across different stages of development, testing, and production, reducing the "it works on my machine" problem.
- **Scalability and Flexibility:** The dual approach of using both CLI and HTTP interfaces allows for flexible interaction with Podman, making it easier to adapt to various deployment scenarios and environments, whether they are local or cloud-based.
- **Error Handling and Reliability:** The use of the `Result` class for encapsulating success and error messages helps in maintaining robustness and reliability in automated processes.
- **Integration with CI/CD Tools:** The Podman integration can be easily combined with CI/CD tools like Jenkins, GitLab CI, or GitHub Actions, facilitating seamless automation of the build and deployment processes.
- **Automation of Builds and Deployments:** The integration of Kubernetes allows for automated creation, management, and deployment of containerized applications, a core aspect of CI/CD pipelines.
- **Environment Consistency:** By using containerization and orchestration, our application ensures consistent environments across different stages of development, testing, and production, reducing the "it works on my machine" problem.
- **Scalability and Flexibility:** Kubernetes provides the scalability and flexibility required to adapt to various deployment scenarios and environments, from local development clusters to large-scale cloud-based systems.
- **Error Handling and Reliability:** The use of the `Result` class for encapsulating success and error messages helps maintain robustness and reliability in automated processes.
- **Integration with CI/CD Tools:** Kubernetes integration can be easily combined with CI/CD tools like Jenkins, GitLab CI, or GitHub Actions, facilitating seamless automation of the build and deployment processes.

3.7 Challenges and Solutions

During the implementation of all mechanisms, the primary challenge was simplifying the work for developers using them. To address this, we created generic classes and functional interfaces to enhance usability (examples to be provided). A significant hurdle was the authentication process, which involved several steps:

- Creating registry headers in `PodmanHttp` and configuring `imageSecrets` for Kubernetes, including encoded username and `accessToken` for image pulling.
- Managing the Kubernetes cluster token, also encoded, by storing it as environment variables and executing a command to retrieve the Kubernetes token for the `kubectl` service account secret.

Our solution involved meticulous planning and organization:

- Storing sensitive information as environment variables ensured security and flexibility in deployment configurations.
- Early implementation and testing of these authentication mechanisms would have made them more comprehensive and user-friendly.

Time management proved to be another significant challenge, impacting the overall completeness and ease of use of the system. Despite these challenges, our approach allowed us to overcome obstacles and deliver robust solutions for authentication and usability in containerized environments.

3.8 Summary

SVLC integrates Podman and Kubernetes to manage containerized services across diverse environments effectively. The integration encompasses `PodmanCmd` and `PodmanHttp`, providing robust CLI and HTTP interactions for seamless pod and container management, as well as efficient image handling. Additionally, the `Kubernetes` class enhances capabilities through the Kubernetes API, enabling advanced functionalities such as scaling, logging, and deploying resources via YAML definitions.

The implementation emphasizes great result handling to ensure consistent error management, aligning with CI/CD principles essential for automated builds and deployments. Key challenges addressed during implementation include improving developer usability through generic classes and interfaces, implementing secure authentication mechanisms for `PodmanHttp` and Kubernetes, and optimizing time management strategies to deliver comprehensive containerized solutions effectively.

Chapter 4

Conclusion and Future Directions

This chapter synthesizes the findings and outlines future directions for Service Virtualization based on Linux Containers (SVLC), leveraging technologies like Podman and Kubernetes. The research has demonstrated the efficacy of integrating these tools to manage and orchestrate containerized services across diverse environments.

4.1 Summary of Findings

The integration of Podman and Kubernetes within SVLC has yielded several key findings:

1. **Seamless Integration:** SVLC effectively integrates Podman for local container management and Kubernetes for cluster orchestration. This hybrid approach facilitates efficient container handling via both CLI and HTTP interfaces (PodmanCmd and PodmanHttp), while leveraging Kubernetes' API for scaling, logging, and resource deployment.
2. **Robust Error Handling:** Implemented through the Result class, robust error handling aligns SVLC with CI/CD principles, ensuring reliability and consistency in automated workflows.
3. **Developer-Friendly Design:** SVLC prioritizes developer usability with intuitive generic classes and functional interfaces, simplifying interactions and enhancing accessibility.
4. **Security and Authentication:** Secure mechanisms for PodmanHttp and Kubernetes include handling registry headers, configuring imageSecrets, and managing cluster tokens securely as environment variables, ensuring robust authentication and authorization protocols.
5. **CI/CD Integration:** Integration with CI/CD pipelines underscores SVLC's automation capabilities, streamlining builds and deployments while leveraging Kubernetes for scalability and environment consistency.

4.2 Future Work

Building on current achievements, future enhancements and explorations include:

1. **Enhanced Security Measures:** Implementing advanced authentication protocols and encryption methods to enhance data security during transit and at rest.
2. **Advanced Orchestration Features:** Expanding Kubernetes class capabilities to encompass advanced features such as network policies, persistent storage management, and service mesh integration.
3. **Scalability Testing:** Conducting comprehensive scalability tests across different deployment scenarios to optimize performance and resource utilization.
4. **User Interface Development:** Developing a graphical interface to complement CLI interactions, enhancing usability for a broader user base.
5. **Integration with CI/CD Tools:** Exploring integration with a wider range of CI/CD tools to broaden SVLC's applicability and flexibility.
6. **Standard Configuration File:** Establishing a standardized configuration file for SVLC to streamline deployment and management across various environments.
7. **Integration with ISOS Framework:** Exploring integration with the ISOS framework to enhance service virtualization capabilities and management flexibility.

4.3 Conclusion

The SVLC project showcases the potential of Linux containers for effective service virtualization. By integrating Podman and Kubernetes, the system provides a robust solution for managing containerized services across diverse environments. The research emphasizes solid error handling, secure authentication, and developer-friendly design as crucial elements in modern container management solutions.

Looking forward, ongoing development and innovation in SVLC will play a pivotal role in advancing container orchestration and management practices in cloud-native environments. The findings and recommendations outlined herein serve as a foundation for future research and development, contributing to the evolution of containerized application deployment and management strategies.

References

- [1] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2, 2014.
- [2] James Turnbull. *The Docker Book: Containerization is the new virtualization*. Lulu.com, 2014.
- [3] Microsoft. Windows containers. <https://docs.microsoft.com/en-us/virtualization/windowscontainers/about/>, 2023. Accessed: 2023-07-12.
- [4] Microsoft. Containers on windows. <https://docs.microsoft.com/en-us/virtualization/windowscontainers/>, 2023. Accessed: 2023-07-12.
- [5] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70, 2016.
- [6] Opencontainers.org. Open container initiative (oci). <https://opencontainers.org/>, 2024.
- [7] M. Armbrust, A. Fox, R. Griffith, A.D. Joseph, R.H. Katz, A. Konwinski, G. Lee, D.A. Patterson, A. Rabkin, I. Stoica, and M. Zaharia. A view of cloud computing, communications of the acm. *ACM Journals*, 2010.
- [8] Andrew S. Tanenbaum. A distributed operating system. *Operating Systems Review*, 19(4):25–32, 1985.
- [9] Carlos Gonçalves, Tiago Dias, A. Osório, and Luis Camarinha-Matos. *A Multi-supplier Collaborative Monitoring Framework for Informatics System of Systems*, pages 44–53. Springer, 09 2022.
- [10] A. Osório, A. Belloum, Hamideh Afsarmanesh, and Luis Camarinha-Matos. Agnostic informatics system of systems: The open isos services framework. In *Collaboration in a Data-Rich World*, pages 407–420, 09 2017.
- [11] Red Hat. What’s a linux container? <https://www.redhat.com/en/topics/containers/whats-a-linux-container>, 2022. [Online; accessed 14-March-2024].

- [12] Sphinx Technologies. What is podman? <https://docs.podman.io/en/latest/>, 2019. [Online; accessed 14-March-2024].
- [13] Daniel J. Walsh. *Podman In Action - The next generation of container engines*. Manning, 2023.
- [14] Phil Hopkins. Vm, container, cloud: Overview, history and differentiation. https://events19.linuxfoundation.org/wp-content/uploads/2017/11/VM_Container_Cloud-Overview-History-and-Differentiation-Talk-Phil-Hopkins-The-Linux-Foundation-.pdf, 2017. Accessed: 2023-07-12.
- [15] Benjamin Hindman, Andrew Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. *USENIX Association*, 44(2), 2011.
- [16] John Clingan and Ken Finnigan. *Kubernetes Native Microservices - with Quarkus and MicroProfile*. Manning, 2022.
- [17] Daniel Sanche. Kubernetes 101: Pods, nodes, containers, and clusters. <https://medium.com/google-cloud/kubernetes-101-pods-nodes-containers-and-clusters-c1509e409e16>, 2018. [Online; accessed 16-April-2024].
- [18] Martin Fowler. Continuous integration. <https://martinfowler.com/articles/continuousIntegration.html>, 2006.
- [19] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [20] Mazin Y. Shahin, Peng Liang, and Mohammad A. Babar. Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 2017.
- [21] The Helm Authors. Helm - the kubernetes package manager. <https://helm.sh/docs/>, 2023. Accessed: 2023-07-13.
- [22] GitLab. What is ci/cd? <https://about.gitlab.com/topics/ci-cd/>. [Online; accessed 17-March-2024].
- [23] fabric8.io. kubernetes-client. <https://github.com/fabric8io/kubernetes-client>.